

STRING_SPLIT vs WHILE Loop CSV Parsing

#sql-server #performance #set-based #refactoring

What the Pattern Looks Like

A stored procedure accepts a comma-separated list as a single string parameter, then parses it into a working table using a WHILE loop:

```
Declare @RestrictAddrBank Table (AddrBank varchar(2))

If (Select Substring(@AddrBankFilter, Len(@AddrBankFilter), 1)) != ','
    Set @AddrBankFilter = @AddrBankFilter + ','

While CharIndex(',', @AddrBankFilter) > 0
Begin
    Insert Into @RestrictAddrBank
    Values (Left(@AddrBankFilter, CharIndex(',', @AddrBankFilter) - 1))

    Set @AddrBankFilter = SubString(@AddrBankFilter,
                                    CharIndex(',', @AddrBankFilter) + 1,
                                    Len(@AddrBankFilter))
End
```

This is procedural code masquerading as SQL. Each iteration allocates a new string for @AddrBankFilter, evaluates CharIndex twice, and inserts a single row. For a CSV with N values, the proc executes N inserts, N+1 string slices, and 2N+1 CharIndex calls. For small N this is cheap, but the engine has no way to optimize the loop; it executes exactly as written.

Why It Hurts

Three issues compound:

- **No set-based optimization is possible.** The engine processes each comma in sequence. There is no parallelism, no batch insert, no plan-cache benefit beyond the single-row INSERT plan.
- **String allocation pressure.** Each iteration produces a new copy of @AddrBankFilter shortened by one token. For a 100-token filter, that is 100 string allocations of decreasing length.
- **The trailing-comma fix-up is brittle.** The If ... Set @AddrBankFilter = @AddrBankFilter + ',' block exists only to make the loop terminate. It is procedural cruft that has nothing to do with the proc's purpose.

For a typical CSV size of 1 to 20 tokens this is invisible in a profile. The pattern matters because it is used inside procs that run millions of times per month; the cumulative cost is real, and the maintainability cost is constant regardless of scale.

The Set-Based Replacement

STRING_SPLIT was added in SQL Server 2016 and has been the canonical replacement since. It returns a single-column table of tokens:

```
Insert Into #RestrictAddrBank (AddrBank)
Select Distinct LTrim(RTrim(value))
From String_Split(@AddrBankFilter, ',')
Where LTrim(RTrim(value)) > ''
```

One INSERT, one SELECT, set-based. The trim and the empty-token filter are optional but defensive: callers that pass 'BR, BL' (with a space) or 'BR,,BL' (a stray double-comma) get the same result they probably expected.

A few practical notes:

- STRING_SPLIT is enabled at compatibility level 130 and higher. For older databases, a tally-table or recursive-CTE splitter has the same set-based shape.
- The output column is named value. There is no ordinal column until SQL Server 2022, so if call order matters, use STRING_SPLIT(@s, ' ', 1) (third argument enables ordinal) or fall back to a custom splitter.
- STRING_SPLIT does not handle quoted values or escaped delimiters. If the input is a true CSV with embedded commas inside quoted fields, use a CLR splitter or pass the parameter as a TVP instead.

Even Better: Table-Valued Parameters

When the caller is application code, the cleanest path is not to pass a CSV at all. A table-valued parameter (TVP) lets the caller pass a real rowset:

```
Create Type AddrBankList As Table (AddrBank varchar(2) Not Null Primary Key)
Go
```

```
Alter Procedure lsp_OrdGetRxStatusCounts
    @AddrBankList AddrBankList ReadOnly,
```

The caller builds a DataTable (or equivalent) and passes it directly. No string parsing, no CSV ambiguity, no quoting concerns. TVPs are read-only inside the proc and carry statistics like a temp table.

TVPs are the right answer for new procs. For existing procs with established callers, the migration cost is real, and STRING_SPLIT is usually the correct interim step.

When to Migrate

Use STRING_SPLIT when you find a WHILE-loop CSV parser inside a proc that:

- Runs frequently (any proc in the top 100 by exec count is worth fixing)
- Uses the parsed list inside a JOIN or EXISTS against a large table (the parsed list is more useful as a temp table with statistics than as a table variable, regardless of how it was populated; see [Table Variables vs Temp Tables](#))
- Was written before SQL Server 2016 (the pattern is everywhere in legacy code)

Skip the migration when the parser is in code you do not own and the proc is not on the offender list. The change is mechanical, but every proc edit has deployment risk and the win is small unless the proc is frequently called.

What Else to Fix at the Same Time

CSV parsing patterns rarely live alone. When a proc parses a CSV into a table variable, three other anti-patterns are likely nearby:

- The table variable should be a temp table. See [Table Variables vs Temp Tables](#).
- The EXISTS predicate that consumes the parsed list often has the [Ambiguous Self-Comparison Predicates](#) bug, because the parsed-list column is named the same as the column it is filtering against.
- The proc may also have catch-all patterns for “no filter means all rows,” which combine with the CSV parser into a multi-issue cleanup. See [Catch-All Query Anti-Pattern](#).

When you find one, look for the others.

Related Concepts

- **Table Variables vs Temp Tables:** where to put the parsed values once they are extracted.
- **Ambiguous Self-Comparison Predicates:** a class of bug that frequently lives in the EXISTS predicate that consumes the parsed list.
- **Catch-All Query Anti-Pattern:** an adjacent pattern where one parameter (often the same CSV) tries to accept “all values” via NULL or empty-string checks.